The Four Build Assessments

Product Engineering Bootcamp

These four assessments are compulsory. Every one of you must complete all four.

Deadline: Wednesday, 17 September 2026. That is two weeks from today. All four repositories, all four documentation files, and all four LinkedIn posts must be submitted by then. Do not leave them for the final weekend. Assessment 2 alone will take you the better part of three days if you do it properly, so start now and work through them in order.

Read this whole page before you begin any of them.

How These Four Work

You are not building four applications. You are building four **slices**, one per assessment. A slice is a single working flow, built properly, with nothing around it.

This is deliberate. A full application lets you hide weak work behind features that look impressive. A single flow does not. If your authentication slice is the only thing in the repository, then your authentication is the only thing being assessed, and it has to hold up.

Each assessment has three deliverables and all three are required:

1. **A GitHub repository** containing the working slice
2. **A documentation file** inside that repository, named `DOCUMENTATION.md`, written to the structure given below
3. **A LinkedIn post** about what you built and the concepts behind it

The documentation carries as much weight as the code. A perfect implementation with a thin document does not pass, because the point of these assessments is to prove you understand what you built rather than that an agent produced it. You will use agents. That is expected. The document is where you demonstrate that you know what came out.

What "do not build" means

Every brief has a **Do not build** section. Take it literally.

Do not build a landing page. Do not build a marketing page. Do not design a full dashboard. Do not add features that were not asked for. Where a brief says the flow ends at a dashboard, that dashboard is a nearly empty page that says "You are signed in" and shows the user's name. That is the whole dashboard.

Every hour you spend on something outside the brief is an hour not spent on the part being graded, and it makes your repository harder to review, which works against you.

The stack

Use the same stack you have been working in. These are slices from the kind of product you already know how to build, not an excuse to learn a new framework.

The Documentation Template

This structure is used for all four assessments. The section headings stay the same every time. What changes is which concepts you are explaining inside them.

Create a file called `DOCUMENTATION.md` at the root of your repository. Use these eight sections in this order.

Section 1: What This Is

Two paragraphs, no more.

The first paragraph says what the slice does, in plain language, as if explaining to a competent person who has not seen your code. The second says what is deliberately not included and why.

Someone should be able to read these two paragraphs and know exactly what they are about to look at.

Section 2: How To Run It

Numbered steps from a fresh clone to a working local instance.

- What to install
- What environment variables are needed, listed by name with a note on where each one comes from
- The database setup and migration command
- The command that starts it
- The URL where it appears

Include an `.env.example` file in the repository with commented placeholders. Never commit real keys. A reviewer who cannot run your project in under ten minutes will assume it does not run.

Section 3: The Flow, Step By Step

Walk through what happens from the user's first action to the last, in order.

For each step state three things: what the user does, what the frontend sends, and what the server does with it. Name the actual route or file where each step lives, so the reader can follow along in the code.

This section is a narrative, not a list of endpoints. A reader should finish it able to predict where in the codebase any given behaviour lives.

Section 4: The Data Model

Show the schema for every table your slice uses.

For each table, explain in one line what it holds. For each column that carries a decision, explain the decision. Why that type. Why that constraint. Why nullable or not nullable.

Then answer this question explicitly: **which constraints in this schema make an invalid state impossible?** A unique constraint, a check constraint, or a foreign key is not decoration. It is the last line of defence when your application code has a bug, and naming what each one prevents shows you chose it rather than accepted it.

Section 5: The Concepts

This is the heart of the document and the most heavily graded section.

Each brief gives you a list of concepts you must cover here. Give every concept its own subheading, and answer four questions for each one, in this order:

What it is. Define it in plain language, in two or three sentences, as though the reader has never heard the term. Not a dictionary definition. Your own words.

Why it is needed. What goes wrong without it. Be concrete and name the failure. "So the app is secure" is not an answer. "Without it, someone can send ten thousand login attempts a minute and eventually guess a password, and each attempt costs me a database query" is an answer.

How I implemented it. What you actually did, with the file or function named, and a short code excerpt where a code excerpt helps. Ten lines maximum per excerpt. If the excerpt needs more than ten lines to make the point, explain it in prose instead.

What I chose against, and why. The alternative you did not take and the reason. This is the question that separates people who made decisions from people who accepted defaults. If you genuinely had no alternative, say so and explain why the choice was forced.

Worked example, so the depth expected is clear:

Password Hashing

What it is. Hashing turns a password into a fixed-length string that cannot be reversed back into the original. When a user signs in, I hash what they typed and compare it against the stored hash. The real password is never stored anywhere in my system.

Why it is needed. If my database is ever read by someone who should not have it, plain passwords would hand that person every account immediately, and because people reuse passwords, it would hand them accounts on other services too. Hashing means a stolen database gives an attacker a set of strings that are expensive to work backwards from.

How I implemented it. I used bcrypt with a cost factor of 12, in `lib/auth/password.ts`. The hash is generated at signup and on password reset, and compared at signin. The cost factor is deliberately

slow, which is the point: a slow hash barely affects one honest login and severely slows an attacker testing millions.

What I chose against, and why. SHA-256 is a general-purpose hash and is fast, which makes it wrong for passwords for exactly that reason. Argon2 is a defensible alternative and arguably stronger, but bcrypt is well supported in my stack and well understood, and choosing the option I can reason about beat choosing the newer one I could not.

Every concept in your list gets that treatment. Four questions. No skipping the fourth.

Section 6: What Went Wrong

A minimum of three problems you hit while building, each with:

- **The symptom.** What you saw.
- **The investigation.** What you checked, including the things you checked that turned out to be irrelevant.
- **The cause.** What was actually wrong.
- **The fix.** What you changed.

Do not sanitise this section. The wrong turns are the part that proves you did the work. A document with no problems in it reads as either untrue or as work you did not do yourself, and reviewers notice both.

Section 7: What This Slice Does Not Handle

An honest list of the limitations you know about.

What breaks at scale. What you would need to add before real users touched it. What you left out because it was outside the brief, and what you left out because you ran out of time. Distinguish between those last two.

This section is not a weakness. Knowing where your own work ends is a senior trait, and a reviewer trusts a document more when it contains one of these.

Section 8: If I Built This Again

One paragraph. The single biggest thing you would do differently, and why.

Not a list. One thing, chosen deliberately.

The LinkedIn Requirement

Every assessment ends with a public post. Same rules each time.

Length: somewhere between 200 and 400 words. Long enough to say something, short enough to be read.

Structure that works:

1. Open with the problem or the thing that surprised you. Not "I just completed assessment two of my bootcamp."
2. Name what you built in one sentence.
3. Explain one concept properly. One. Pick the most interesting thing you learned and teach it in three or four sentences, the way you would explain it to a colleague.
4. Include a real detail with a number or a specific behaviour in it.
5. Link the repository.

What to avoid:

Do not write a progress update. "Week 6 done, learning so much" tells a reader nothing and is indistinguishable from a thousand other posts.

Do not list technologies. "Built with Next.js, Prisma, PostgreSQL, TypeScript" is a stack, not an insight.

Do not pretend it was easy, and do not pretend it was harder than it was. Both read as performance.

The test: would someone who does not know you learn something from this post? If the only information in it is that you completed a task, rewrite it.

ASSESSMENT 1

The Authentication Slice

Time budget: 14 to 18 hours

What to build

A complete authentication system with its own interface, ending at a placeholder dashboard.

Screens: - Create account - Sign in - Forgot password, meaning the request form - Reset password, meaning the form reached from the emailed link - Email verification, including the code entry and the resend control - A placeholder dashboard that shows the signed-in user's name and a sign out button, and nothing else

Behaviour: - A new user can create an account, verify their email, and reach the dashboard - A returning user can sign in - A user who forgets their password can reset it and sign in with the new one - A signed-out user who types the dashboard URL directly is sent to sign in - Sign out ends the session properly

Do not build

No landing page. No marketing page. No dashboard features. No profile editing, no settings, no social sign-in, no two-factor. The dashboard is one line of text and a button.

Engineering requirements

These are the things being assessed. Every one of them must be present and must appear in your documentation.

- **Password hashing** with an adaptive algorithm, not a general-purpose hash
- **Server-side validation** on every input, declared as a schema rather than scattered through handlers, with client-side validation mirroring it for feedback
- **Rate limiting** on signin, signup, password reset request, and verification code resend
- **Session management**, with the session cookie configured correctly
- **Email verification codes that expire in the database**, not only in the interface
- **A resend cooldown enforced on the server**
- **Password reset tokens** that are single use and time limited
- **A unique constraint on email** at the database level
- **An idempotent signup endpoint**, so a double submission creates one account
- **Protected route handling**, so the dashboard cannot be reached without a session
- **Input groups with labels programmatically bound to inputs**, and visible focus states

Concepts to document in Section 5

Password hashing. Rate limiting. Client-side versus server-side validation. Session management and why you chose sessions or tokens. Token and code expiry, and why expiry must live in the database. Idempotency. Database constraints as a last line of defence. Protected routes.

Prove it works

Your documentation must include evidence, not claims:

- A screenshot of the users table showing a stored hash, so it is visible that no plain password exists
- The exact curl command you used to hit your signup endpoint directly, bypassing the browser entirely, and what the server returned
- Evidence of the rate limit triggering, showing the status code returned
- A screenshot of a verification code in the database, and the same record after expiry

Grading bands

Pass: all screens work, all engineering requirements present, documentation complete with all eight sections.

Excellent: validation rules are declared once and shared between client and server; the curl evidence shows the server rejecting what the browser would have blocked; Section 5 answers the fourth question properly for every concept; the rate limit returns a correct status code with a retry indication.

Traps

Validating only on the client and assuming that is validation. Building the dashboard instead of the authentication. Leaving verification codes with no server-side expiry, so the interface countdown is theatre.

Committing your `.env` file. Rate limiting the signin route and forgetting the resend route, which is the one that costs you money.

Defence questions

Your reviewer will ask you these. Prepare for them.

- Why that hashing algorithm, and what happens if I set the cost factor to 4?
- Show me the exact line where the session is created and tell me what is inside the cookie.
- I send your signup request twice in the same second. Walk me through what happens in the database.
- Which of your validation rules cannot be enforced on the client, and why?

ASSESSMENT 2

The Payment and Subscription Slice

Time budget: 20 to 26 hours

This is the largest of the four. Budget accordingly.

What to build

A working subscription system in test mode, with one paid plan sold on two intervals, monthly and yearly.

Screens: - A plans view showing free, monthly, and yearly, with the current plan indicated - A checkout initiation that hands off to your payment provider - A return view the user lands on after paying - A billing view showing plan, status, renewal date, and a cancel control - A minimal signed-in shell to hang these on

Behaviour: - A user can subscribe to monthly - A user can upgrade from monthly to yearly mid-cycle, and the amount charged is prorated - A user can downgrade, with the change applied at the end of the current period - A user can cancel, and keeps access until the period they paid for ends - Every payment event is recorded

You may reuse your authentication from Assessment 1 to get a signed-in user. Say so in your documentation if you do. Reuse is not cheating; hiding it is.

Do not build

No landing page. No pricing marketing page. No product features behind the paywall. The thing being sold is a plan flag on a user record and nothing more.

Engineering requirements

- **Money stored as whole numbers in minor units**, with the currency stored alongside

- **A payment log table** recording every event separately: initiation, verification, fulfilment, failure
- **Server-side verification** before any entitlement is granted, and never on the strength of a frontend claim or a redirect alone
- **Webhook handling** with signature verification before any processing
- **Idempotency keyed on the provider reference**, so a repeated webhook is recorded once and acted on once
- **Proration** on a mid-cycle interval change, calculated and shown
- **Cancellation that retains access to the end of the paid period**, with a confirmation step
- **A cancellation reason column**, populated from an optional post-cancellation prompt
- **Rate limiting** on the checkout initiation endpoint
- **Error handling that never leaves a user on a blank page or a 404**, at any point in the payment path
- **No card details stored anywhere in your system**

Concepts to document in Section 5

Minor units and why money is never a decimal. The payment lifecycle of initiation, verification, and fulfilment, and why they are three separate things. The payment log and what it would prove in a dispute. Idempotency in payments. Webhook signature verification. Proration, including your actual calculation shown with numbers. Cancellation and period-end access, including the legal reasoning. Why you do not store cards, naming PCI scope. Rate limiting on payment endpoints.

Prove it works

Database evidence is mandatory throughout this assessment. Claims without screenshots do not count.

- A screenshot of the subscription record before and after an upgrade, showing the interval changed and the period end moved
- A screenshot of the payment log for one complete transaction, showing each stage as its own row with timestamps
- Your proration calculation written out with real numbers: days remaining, credit applied, amount charged, and the ledger or log entries that resulted
- Evidence of firing the same webhook twice, showing the second one recorded and ignored
- A screenshot of a cancelled subscription showing access retained and the period end date

Grading bands

Pass: subscribe, upgrade with proration, downgrade, and cancel all work; the payment log records every stage; entitlement is granted only after server-side verification; database evidence provided for each.

Excellent: the payment log is append-only and entitlement is derived from it rather than stored and mutated; the duplicate-payment case is tested and handled, meaning paying twice for an active plan either extends correctly or is rejected, rather than absorbing the money silently; the proration calculation is correct to the day and shown.

Traps

Storing amounts as decimals. Granting the subscription when the user lands back on your success URL, which means anyone who visits that URL directly gets a free subscription. Cancelling with immediate cutoff after taking payment for the period. Skipping the payment log because the subscription table already shows a status; the status is the present, the log is the history, and the history is what you produce in a dispute. Testing only the happy path and never firing a duplicate webhook.

Defence questions

- Show me the exact line where entitlement is granted, and tell me what happens if I reach that code path directly in my browser.
- A customer disputes a charge from three months ago. What do you show them, and where does it come from?
- Walk me through your proration arithmetic for an upgrade on day 12 of a 30-day cycle.
- I pay for yearly twice in one minute. What does your database look like afterwards?

ASSESSMENT 3

The AI Integration Slice

Time budget: 18 to 22 hours

What to build

A single AI-powered flow: a user uploads something, a background job processes it through a model, and the result appears.

Choose your own domain. Receipts to expense summaries, handwritten notes to structured text, job descriptions to structured requirements, product photos to descriptions. It does not matter what it is. It matters that it runs through a real model, in the background, with structured output.

Screens: - An upload view accepting one or more files, with size and type restrictions enforced - A processing state that honestly reflects what is happening, showing pending, processing, done, or failed - A result view showing the output - One user-triggered follow-up action on the result, such as summarise, rephrase, or expand

Behaviour: - Upload triggers a background job rather than blocking the request - Two models are used, routed by task, or one model with two distinct system prompts serving two distinct roles - The output is structured data, not free text your code has to guess at - Failures are recorded and visible, and the user is told the truth

Do not build

No landing page. No account system beyond what is needed to have a user, and reusing Assessment 1 is fine. No editing, sharing, or exporting features. One flow, done properly.

Engineering requirements

- **Official SDKs only.** Where a provider has none for your platform, use a compatible official SDK pointed at their endpoint, and explain that in your documentation
- **API keys written into `.env` by hand**, never by an agent, with an `.env.example` carrying commented placeholders
- **A configuration file** holding every changeable value: model identifiers, timeouts, output token caps, temperature, rate limits, and concurrency
- **A written system prompt per role**, with each parameter you set justified in one line
- **Structured output**, requested with a schema and validated in your own code on receipt, with a defined retry and a defined graceful failure
- **A job record in the database** for each unit of work, holding status, attempts, and the error message on failure
- **A queue or concurrency cap** so uploading many files does not fire many simultaneous provider calls
- **Rate limiting** on the endpoint that triggers processing and on the follow-up action
- **Files in object storage or a documented local development equivalent**, with only the storage key in the database, never the file itself
- **A timeout on every model call**, with a defined fallback

Concepts to document in Section 5

What an API endpoint is. SDKs versus raw HTTP, and why official SDKs. System prompts versus user prompts. Model parameters, covering the ones you set and why. Structured output and schema validation, including what you do when validation fails. Jobs and workers. Queues, FIFO, and why concurrency is capped. Rate limiting as a cost control. Why files live in object storage rather than the database. Your cost model, meaning what one run costs you approximately and what caps the total.

Prove it works

- A screenshot of your jobs table showing a successful run and a failed run, with the error message visible on the failure
- The raw model output for one request alongside your validated, parsed result
- Evidence of what happens when validation fails, produced by deliberately breaking the schema or the response
- Your concurrency cap holding, demonstrated by uploading enough files at once and showing the provider request pattern
- A screenshot showing the database holds only a storage key, not the file

Grading bands

Pass: upload triggers a background job, two roles are served by models, output is structured and validated, failures are recorded, keys and config are handled correctly.

Excellent: validation happens in your own code rather than relying only on the provider's schema enforcement; the failure path is a designed user experience rather than an error string; the cost model in Section 5 has real numbers in it; the concurrency cap is demonstrated rather than asserted.

Traps

Letting an agent write your API keys. Hardcoding the model name and the token limit in the handler. Parsing prose with string operations instead of requesting structured output. Testing with three clean inputs and never with an empty file, a corrupted file, or one at the size limit. Storing the uploaded file in the database. Believing a 200 response means the work succeeded, when the work happens in a job and the job is where the failure lives.

Defence questions

- Justify your temperature setting and your output token cap.
- Show me what a user sees when the provider times out.
- Your model returns something that fails validation. Trace what happens next, line by line.
- I upload fifty files and press the button. What exactly happens, and what stops it costing you fifty simultaneous calls?

ASSESSMENT 4

The Records and Access Slice

Time budget: 14 to 18 hours

What to build

A flow where users create, view, and delete records that belong to them, built so that no user can ever reach another user's data.

The records can be anything: projects, notes, invoices, bookings. The domain does not matter. What matters is ownership, correct access control, and efficient data access.

Screens: - A list of the signed-in user's records, with a true empty state - A create action - A detail view for one record - A delete action with confirmation

Behaviour: - A user sees only their own records, everywhere, without exception - Views change without full page loads, but the URL still updates so any view can be shared or bookmarked - Deletion is recorded for audit before the record disappears - Nothing in the URL or the interface exposes a raw database identifier

Do not build

No landing page. No editing, search, tags, sharing, or collaboration. No dashboard widgets. Create, list, view, delete. That is all.

Engineering requirements

- **Every query scoped to the authenticated user in the query itself**, not checked after fetching
- **No raw database identifiers exposed** in URLs or in the interface
- **An audit record written for every deletion**, capturing who deleted what and when, persisted before or as part of the delete
- **Conditional views with URL state**, so navigation is fast but every view is addressable
- **Correct status codes throughout**, with 401 and 403 used for their distinct meanings
- **A measured query count** for each of your three main actions, documented, with a stated reduction from your first working version
- **Indexes** on the columns you filter and sort on
- **Genuine empty states**, with no placeholder or fake data anywhere

Concepts to document in Section 5

Authentication versus authorisation. Scoping the query versus checking after the fetch, and why the first makes a leak structurally impossible. Insecure direct object references, meaning what happens when a user edits an identifier in a URL. Why raw database identifiers are not exposed. Audit logging and why deletions are recorded. Page architecture, meaning conditional rendering with URL state and why both matter. Status codes, specifically 401 against 403. Database indexing. Query count as a cost, with your before and after numbers.

Prove it works

This assessment is graded largely on evidence of attack and measurement.

- **An access control audit table.** Create two users. For every route, attempt to reach user one's data as user two, by editing identifiers, replaying requests, and calling endpoints directly with curl. One row per route: method, path, what you attempted, what happened, pass or fail. Every row must say pass by the time you submit, and the table must show what you tried, not only the outcome.
- **A query count table**, showing each of your three main actions before and after your reduction, with the classification of what each query was doing.
- A screenshot of the audit log after a deletion.
- A screenshot of a URL showing an identifier that is not the database identifier.

Grading bands

Pass: ownership enforced everywhere, audit table complete with every row passing, deletions logged, URLs update with view changes, no raw identifiers exposed.

Excellent: every ownership rule is enforced by query scoping rather than by post-fetch checks; the query count reduction is meaningful and the classification is accurate; Section 5 identifies the single change that would have prevented the most audit failures.

Traps

Hiding the identifier and believing the record is now protected. Obscurity is not access control; the ownership check is. Adding a check after fetching, which works today and will be forgotten by whoever writes the next route. Logging the deletion after the row is gone, so the audit record cannot reference what was deleted. Building conditional views and forgetting the URL, so nothing can be shared. Testing access control with one user account, which tests nothing.

Defence questions

- Show me a query and tell me what happens if I remove the user condition from it.
- I change the identifier in this URL to a value I guessed. Walk me through every layer that stops me.
- Your query count went from a higher number to a lower one. Which single change did the most, and why?
- Why is this a 403 and not a 404, or the other way round?

Submission Checklist

Run through this before submitting any of the four.

Repository - ☐ The slice runs from a fresh clone using only the steps in Section 2 - ☐ `.env` is not committed. Open the repository in a private browser window and confirm with your eyes, not from memory - ☐ `.env.example` is present with commented placeholders - ☐ Commit history shows incremental work, not one commit - ☐ Nothing outside the brief was built

Documentation - ☐ `DOCUMENTATION.md` is at the repository root - ☐ All eight sections present, in order - ☐ Every required concept has its own subheading in Section 5 - ☐ Every concept answers all four questions, including the fourth - ☐ Section 6 contains at least three real problems with real investigations - ☐ All required evidence screenshots are included and readable

LinkedIn - ☐ Posted, with the repository linked - ☐ Teaches one concept properly rather than announcing completion - ☐ Contains at least one specific number or behaviour - ☐ Passes the test: a stranger learns something from it

Yourself - ☐ You can open any file in the repository and explain why it exists - ☐ You have read the defence questions and answered each one out loud

---

A Note On How These Are Graded

The code proves the thing runs. The documentation proves you understand it. The defence proves the documentation is yours.

All three are required because any two can be produced without the third being true. Weight your effort accordingly, and expect the defence questions to be asked exactly as written.

All four are due by **Wednesday, 17 September 2026**, and all four are compulsory. Submit each one as you finish it rather than holding them all to the last day. If something is blocking you, raise it early while there is still time to help you, not on the deadline.